Week 7: Diffusion and flow matching

Jin-Guo Liu, Xi-Wei Pan

Abstract: How can a computer learn to generate new data from random numbers? We start with points arranged in four groups, called clusters. We train a function to predict random disturbances added to these points, then use its predictions to generate new points. We measure whether they recover the clusters. Next we compare ways to reach the same distribution with fewer steps. Finally, we download a function already trained on CIFAR-10 photographs and use it to generate images and fill in missing pixels. The four-cluster problem has exact answers against which we can check the learned quantities. Each Part's Goal states the result you will judge.

Diffusion models

Generation from random numbers. We have examples of the data we want, such as photographs, but no rule for drawing new ones. A *generative model* learns such a rule from the examples. We use a *neural network*, a function built from successive calculations with adjustable numbers. *Training* adjusts those numbers using examples. The model's goal is to reproduce the data distribution: which outcomes are common and which are rare. Drawing an outcome from a distribution is *sampling*. The outcome is a *sample*. Week 6 assigned an energy $E_\theta(x)$ to each possible sample. Turning the numbers $\exp(-E_\theta(x))$ into probabilities required division by a constant Z_θ, the *normalizer*. Computing it was impractical, so we generated samples by repeated Gibbs updates, changing some variables while holding the others fixed. Successive samples could remain similar because each came from the preceding one. This week asks whether we can generate by transforming fresh random numbers instead.

The target density and Gaussian source. For continuous data, write the desired *probability density* as $p_{\text{data}}(x)$. In one dimension, the probability of an interval is the area under this curve over the interval. The curve can have several peaks, as on the left of Figure 1. We want generated points to occupy each interval with that probability. Our starting numbers come from a density that a computer can readily sample, the *standard Gaussian*. This bell-shaped density is centred at zero. Its *variance*, the average squared distance from zero, is one. Its one-dimensional notation is $\mathcal{N}(0,1)$, and it is the right-hand curve of Figure 1. We draw numbers *independently*: knowing one draw does not change the probabilities for another.

Sampling through an invertible map. Suppose a function f transforms the target distribution into that Gaussian. We call it a *transport map* because it moves the samples from one distribution to another. Suppose every output has exactly one input. We can then undo the function with an *inverse*, f^{-1}. The map is *invertible*. Such a map is also called a *bijection*. We could then draw a fresh Gaussian number z and return $x = f^{-1}(z)$. If the map carries the distribution exactly, these outputs are independent samples of the target. This is the sampling direction in Figure 1. A *normalizing flow* learns the map f directly. Examples are nonlinear independent components estimation (NICE) and real-valued non-volume preserving transformations (RealNVP). They construct f so that it is easy to undo. This requirement limits the functions they can use. Diffusion takes a different approach to reaching an easy-to-sample distribution.

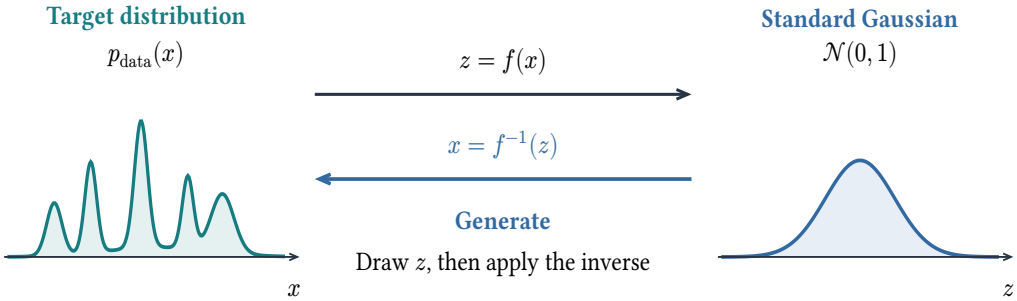


Figure 1: Generation is the inverse of a transport map: the forward map f sends the target distribution to a standard Gaussian, and f^{-1} turns fresh Gaussian draws into target samples. The curves are schematic one-dimensional densities.

Forward diffusion. We can disturb a data point by adding random numbers to its coordinates. These added numbers are noise. Here we draw an independent standard Gaussian number for each coordinate and scale it to set the size of the disturbance. For an image, the coordinates are the numbers storing its pixel colours. A *clean sample* is a data sample before we add this noise. For points we also say *clean point*, and for photographs, *clean image*. "Clean" does not remove the variation already present in the data. Estimating the original sample from its noisy version is *denoising*. To turn data

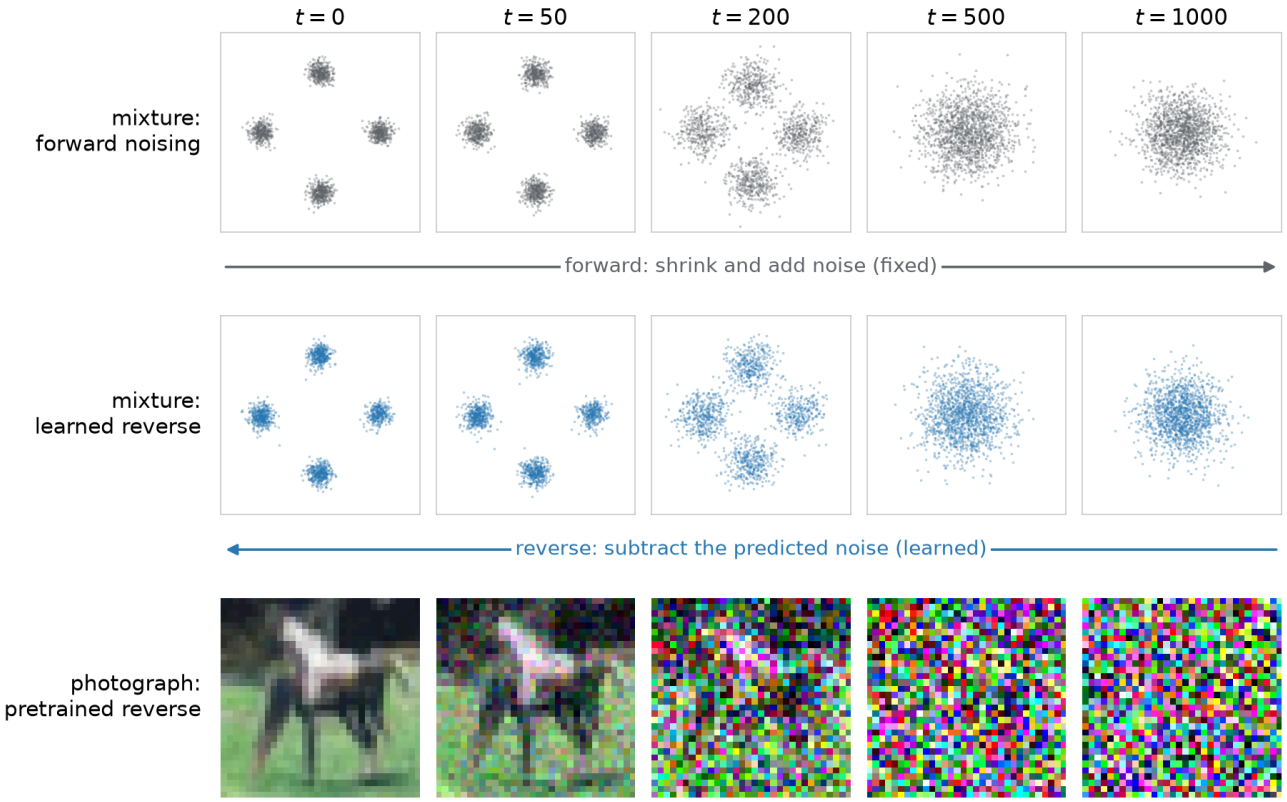


Figure 2: The forward and reverse processes. Top: forward diffusion turns four clusters into an approximately Gaussian cloud. Middle: a learned reverse process turns fresh Gaussian noise back into four clusters. Bottom: a downloaded network guides the reverse process for a 32×32 photograph. Columns use the noise levels defined above. All panels are reference runs from Parts 1 and 4.

into something easy to sample, repeatedly shrink it toward zero and add a little noise. This is forward diffusion. Write the clean sample as x_0. The sample's current values are its *state*, written x_t after t steps. The step index t is its noise level, also called the diffusion level $T = 1000$ is the last level in this week's experiment. The forward rule is fixed. Read the top row of Figure 2 from left to right. By $t = 200$ the four clusters have blurred. By $t = 1000$ the original sample contributes very little and the points form an approximately standard Gaussian cloud. Many clean points can lead to the same noisy point. For generation we learn to move from noise back toward data, the *reverse process*.

Generation by small reverse steps. Producing a whole photograph from random numbers in one step is a difficult learning task. Diffusion replaces it with a sequence of small changes between neighbouring noise levels. Generation starts with fresh Gaussian noise at $t = T$. A learned rule takes it to the next lower level, $t - 1$; repeating the rule reaches a new sample at $t = 0$. The positions along a run form its *path* or *trajectory*. Its first and last points are its *endpoints*. Read the middle and bottom rows of Figure 2 from right to left: clusters reappear, and a horse emerges from static. Early reverse steps establish coarse structure, such as which cluster a point joins. Later steps refine its position within the cluster. On images, this is the progression from broad shapes to detail. The aim is to recover the data's distribution, not the particular sample that started a forward run.

Noise prediction and DDPM sampling. Each reverse step needs information about the noise in its current input. We learn that information from examples with known answers: take a clean sample, add noise, and predict the added noise. The answer the network should predict is its *target*. Fitting numerical targets this way is *regression*. Here the network is a noise predictor because its target is the added noise. A *denoising diffusion probabilistic model* (DDPM) uses the prediction to choose a small reverse move, then adds fresh randomness. The rule for generating samples is the *sampler*. Several earlier states can be consistent with one noisy state. Random reverse steps allow the sampler to produce different possible histories. Part 1 builds this sampler and explains what the network can predict when the clean origin is uncertain.

Fewer-step sampling with DDIM and flow matching. Part 1's DDPM sampler visits all 1000 noise levels. Every step evaluates the network, so generating one sample requires 1000 predictions. This becomes expensive for images. A *denoising diffusion implicit model* (DDIM) reuses the trained noise predictor but changes the reverse update. It can jump between a selected subset of noise levels without retraining the network. We use its deterministic version: after

the starting draw, its steps make no random choice. A sampler that does make fresh random choices during its steps is *stochastic*, as DDPM is. Part 3 also trains a network to say how fast and in which direction each point should move. Its output is a *velocity*. Each training example joins a data point to a random Gaussian point by a straight line, and the velocity along that line is known. This method is flow matching. Following these velocities also makes no random choice between steps. We will measure how few steps each route needs to recover the target.

Origins of diffusion models. Sohl-Dickstein and coauthors [1] introduced diffusion models in 2015. Song and Ermon [2] learned directions toward more likely samples at different noise levels in 2019. Ho, Jain and Abbeel [3] introduced the DDPM noise-prediction training rule in 2020. These papers develop the generation problem through different choices of what the network learns.

Training and generation in the five Parts. The roadmap separates training from generation. Part 1 trains the noise predictor on four clusters. Parts 2 and 3 compare ways to generate samples with fewer steps. Parts 4 and 5 use a noise predictor that someone else has already trained on photographs, first to generate whole images and then to fill missing pixels.

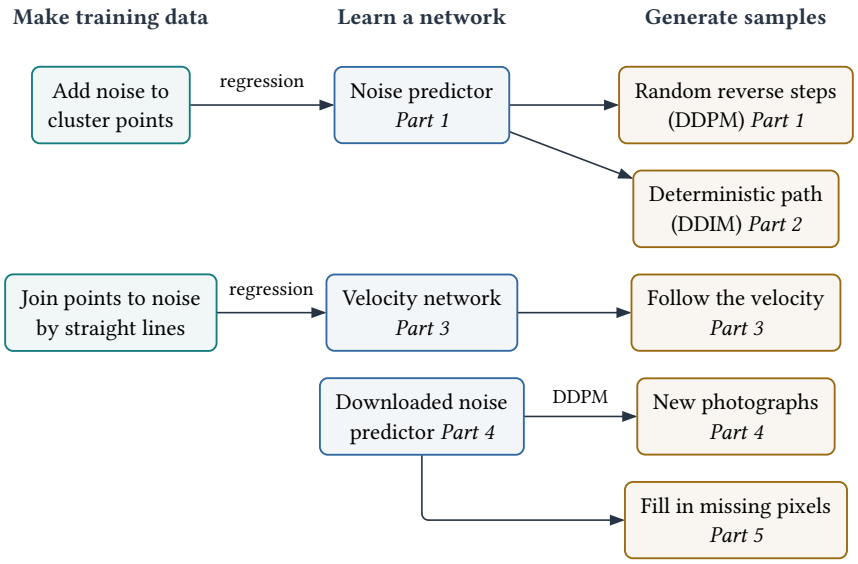


Figure 3: Training and generation have different jobs. Training fits a network to examples with known answers. A sampler then uses that network to generate new data. Parts 1 to 3 work on four clusters, where the learned quantities have exact counterparts. Parts 4 and 5 work on photographs.

Resources:

- Niels Rogge and Kashif Rasul, *The Annotated Diffusion Model*. Parts 1 and 4 follow its noising, noise-prediction and reverse-sampling sequence, using the numerical-computing library JAX rather than PyTorch.
- Ho, Jain and Abbeel, *Denoising Diffusion Probabilistic Models*. Algorithms 1–2 are the training and sampling loops of Parts 1 and 4.

Part 1: Diffusion models on a Gaussian mixture

Goal: train a noise predictor and use it to generate new samples. Check that the samples form the four target clusters, with the right fraction of points in each cluster and the right centres and widths.

① DO

Part 1 builds a diffusion model for the four clusters. The idea is to turn the clusters into noise in small steps, then learn to undo those steps. The surprising part is that the network never learns to generate: it only learns to spot the noise we added, and that is enough to draw new points. No normalizer like Week 6's is needed. Applying the undo rule one step at a time, starting from fresh noise, produces new samples.

The prompts below set up this model, train it and run it.

Work in `week7/` inside your existing exercise repository. Your laptop's central processing unit (CPU) is the baseline. Send the prompts, then read UNDERSTAND while your agent works.

```
Work in week7/ using uv, with CPU as the default. Save figures in evidence/ and raw
runs with settings in artifacts/. Keep downloads and raw runs out of git. Commit after
each task; ask about uncertainties instead of guessing.
```

```
Build forward diffusion in JAX for an equally weighted Gaussian mixture with centres
(3,0), (0,3), (-3,0), (0,-3) and covariance 0.09 times the identity. Use 1000 steps
with per-step noise variance beta_t increasing linearly from 0.0001 to 0.02, where t
is the diffusion level. Implement DDPM sampling with variance beta_t, no clipping and
no added noise in the final step.
```

```
Train a small network to predict the added noise. Make the predictor time-conditioned.
Minimize mean squared noise-prediction error over uniformly sampled noise levels.
Decay the learning rate to zero by the end of training. Record training settings and
loss history. Retain the model at 10 training stages, including initialization and the
final model.
```

If your agent asks about the training length: the reference makes 25000 updates using 512 newly drawn points per update, about one minute on a laptop CPU. It saves models more frequently near the start, when learning changes the predictions most.

(2) UNDERSTAND

Part 1 builds four objects in order: the target, the forward noising process, the network that predicts the noise, and the reverse updates that turn predictions into samples.

Probability density and expected values. Our sample has d real coordinates. Such a list of numbers is a *vector*, written $x \in \mathbb{R}^d$. The opening's area-under-a-curve rule extends to a density $p(x)$ in d dimensions: integrate over a region to find its probability. Integrating over the whole space gives one. A density value at one point is not itself a probability. To compare generated points with the target, we need averages. The *expected value* of a quantity $h(x)$ weights it by the density:

$$\mathbb{E}[h(x)] = \int h(x)p(x)\,\mathrm{d}x. \tag{1}$$

A sample average estimates this integral from many draws. For example, $\mu = \mathbb{E}[x]$ is the mean point, or centre of the distribution. The mean of a vector takes the average of each coordinate separately.

Variance, standard deviation and covariance. A mean does not tell us how widely the points spread. The variance of coordinate x_i is its average squared departure from the mean, $\mathbb{E}\left[(x_i - \mu_i)^2\right]$. Its square root is the *standard deviation*, measured in the same units as x_i. Two coordinates may also vary together. Their *covariance* is $\Sigma_{ij} = \mathbb{E}\left[(x_i - \mu_i)(x_j - \mu_j)\right]$. The diagonal entries of Σ are the coordinate variances. Off-diagonal entries describe how two coordinates vary together. This matrix records the spread of a cloud in all directions.

Gaussian clusters and Gaussian noise. A Gaussian density is fixed by its mean μ and covariance Σ; we write it as $\mathcal{N}(x; \mu, \Sigma)$. Our target clusters use $\Sigma = \sigma^2 I$. The *identity matrix* I has ones on its diagonal and zeros elsewhere, so multiplying by it leaves a vector unchanged. Each cluster's coordinates are independent Gaussian variables with the same standard deviation σ. This gives a round cloud centred at μ. For the noise we use $\mu = 0$ and $\Sigma = I$. The notation $z \sim \mathcal{N}(0, I)$ means to draw such a random vector. Each coordinate has mean zero and variance one.

The Gaussian-mixture target. To give generation more than one destination, combine several Gaussian clusters. This is a Gaussian mixture. Each Gaussian is a *component* of the mixture. Its density is

$$p_{\text{data}}(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x; \mu_k, \Sigma_k), \quad \pi_k \geq 0, \quad \sum_{k=1}^{K} \pi_k = 1. \tag{2}$$

Here K is the number of components and π_k is the probability of choosing component k. To draw a point, choose the component first and then draw from its Gaussian. These are *exact samples*, draws directly from the prescribed target distribution.

Our target has four equally likely components in two dimensions. Thus $K = 4$, $d = 2$ and $\pi_k = \frac{1}{4}$. Their covariance is $\Sigma_k = 0.09I$, and their means are

$$\mu_1 = (3, 0), \quad \mu_2 = (0, 3), \quad \mu_3 = (-3, 0), \quad \mu_4 = (0, -3). \tag{3}$$

The coordinate standard deviation is $\sqrt{0.09} = 0.3$. This is small compared with the distance between centres, so the density has four separated modes, regions of high density. A clean point is a random draw within one cluster; it need not be the centre.

Mode populations, centres and widths. A sampler might reach all four clusters but visit one too often. Assign every generated point to its nearest target mean. A group's *population* is its fraction of all generated points; each should be close to 0.25. Correct populations can still hide misplaced or broadened clusters, so also measure each group's centre. This is the average of its points. Its *per-coordinate spread* is the standard deviation of each coordinate within the group. Compare the centres with the four target means and the spreads with 0.3.

The forward diffusion update. We now write the opening's forward rule as a numerical update. The diffusion level runs through $t \in \{1, \ldots, T\}$, with $T = 1000$. Let β_t be the variance of the noise added to each coordinate in step t, and set $\alpha_t = 1 - \beta_t$. The update is

$$x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{\beta_t} z, \quad z \sim \mathcal{N}(0, I). \tag{4}$$

It shrinks the previous state and adds a fresh Gaussian disturbance. Adding noise alone would make the cloud spread indefinitely. The shrinkage reduces its variance by a factor α_t, while the independent added noise contributes β_t. Because $\alpha_t + \beta_t = 1$, a coordinate with variance one keeps variance one under this update.

The variance schedule and noisy densities. The list of per-step variances $\beta_1, \ldots, \beta_T$ is the *variance schedule*. This week's schedule grows linearly from $\beta_1 = 10^{-4}$ to $\beta_T = 0.02$. Early steps disturb the data gently; later steps add more noise to an already noisy cloud. Write $q_t(x)$ for the density of the samples after t forward steps. At $t = 0$ it is the target density, and at high levels it approaches the standard Gaussian. This density comes from the fixed noising process, independently of the learned model.

Direct noising to any level. Training will need examples at many noise levels. Running all the preceding steps for each example would be wasteful. We can combine them into one draw because a sum of independent Gaussian noises is Gaussian. Define $\overline{\alpha}_t = \prod_{s=1}^{t} \alpha_s$, with $\overline{\alpha}_0 = 1$. The result of the t steps has the same distribution as

$$x_t = \sqrt{\overline{\alpha}_t} x_0 + \sqrt{1 - \overline{\alpha}_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I). \tag{5}$$

Here ε is the single noise vector for this direct draw. A training example at any level therefore costs one draw rather than t steps. At $t = 1000$, $\overline{\alpha}_t \approx 4 \times 10^{-5}$. The coefficient of the clean point in Equation 5 is almost zero, and the noise coefficient is almost one. This explains why the four clusters have become an approximately standard Gaussian cloud.

The time-conditioned noise predictor. To estimate the noise in x_t, the network receives the noisy point. It does not receive the clean point x_0, which is unavailable during generation. The same point can require different predictions at different levels, so we also give it t. A network that receives the level is time-conditioned the step index is often called *time*. We write its output as $\varepsilon_\theta(x_t, t)$, with θ denoting the network's adjustable numbers. It has two coordinates, one estimated noise value for each coordinate of the point.

The predictor's architecture and weights. To process the noise level alongside the point, the reference network computes a list of numbers from t. These numbers are its *time features*, a representation of the noise level. The combined inputs pass through a sequence of intermediate calculations, called *hidden layers*. Each layer transforms the preceding layer's numbers. In our demo, each value can depend on every value in the preceding layer; this makes the layer *fully connected*. The last layer produces the two predicted noise coordinates. The arrangement of layers is the network's architecture, shown in Figure 4. The adjustable numbers θ are its weights, also called *parameters*. Giving them their starting values is *initialization*; we start from random values. Training changes the weights while keeping the architecture fixed. Part 3 reuses this architecture to predict velocity, but trains a separate set of weights.

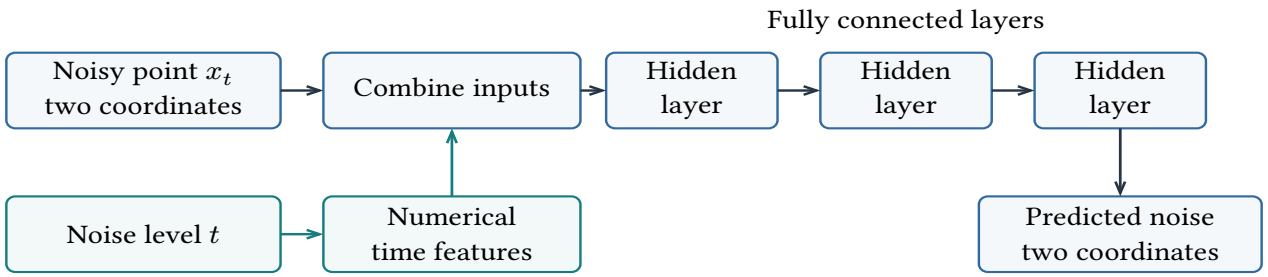


Figure 4: The Gaussian-mixture noise predictor. The noisy point and a numerical representation of its noise level enter the same small network. Three fully connected hidden layers lead to a two-coordinate noise prediction. This is one prediction, not a sequence of sampling steps.

Training examples with known noise. We use Equation 5 to construct x_t from a clean point and a known noise draw ε. That draw is the training target. As during generation, the network receives only (x_t, t). Figure 5 shows how one clean point supplies training examples at different noise levels.

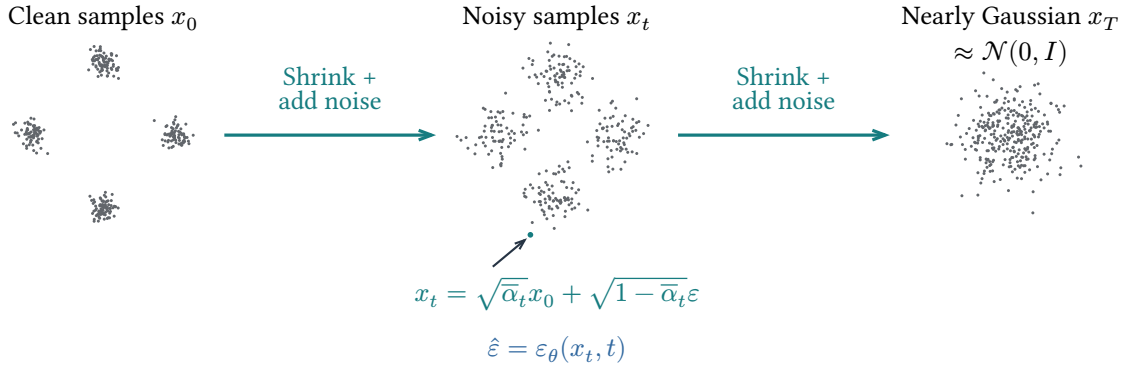


Figure 5: Adding noise supplies the training target: at every level the clean sample is shrunk and noise is added, and the network must return that noise. The same 300 reference samples are shown at $t = 0, 200, 1000$ on one scale; the arrow follows one sample. The teal equation constructs x_t from clean x_0 with known noise $\varepsilon \sim \mathcal{N}(0, I)$; the blue equation predicts that noise from (x_t, t).

The noise-prediction loss. To judge a prediction, subtract the known noise and square each coordinate's error. For a vector w, its length, or *norm*, is $\|w\| = \sqrt{\sum_i w_i^2}$. Thus $\|w\|^2$ is the sum of its squared coordinates. The mean squared noise-prediction error is

$$\mathcal{L}_{\text{noise}}(\theta) = \mathbb{E}_{x_0, t, \varepsilon}\left[\frac{1}{2}\|\varepsilon_\theta(x_t, t) - \varepsilon\|^2\right]. \tag{6}$$

The expectation averages over clean points, noise levels and Gaussian noise. We draw levels *uniformly*, giving every level the same probability. The factor $\frac{1}{2}$ averages over the two coordinates. This quantity is the training *loss*, the number we try to reduce.

Updating the weights during training. Each update estimates the loss on a *batch*, a group of fresh examples. The loss has a derivative with respect to each weight; the vector of these derivatives is its *gradient*, and it points toward increasing loss. JAX computes it by *automatic differentiation*, as in Week 5. The box's gradient step moves against it, and the *learning rate* sets the step size. Reducing it toward zero lets the final updates make smaller adjustments. Saving a copy of the weights preserves a *training stage*, so we can compare predictions before, during and after learning.

The training loop is:

function TRAIN-NOISE-PREDICTOR(θ)
  for each update **do**
    $x_0 \sim p_{\text{data}}$
    $t \sim \mathcal{U}\{1, \ldots, T\}$
    $\varepsilon \sim \mathcal{N}(0, I)$
    $x_t \leftarrow \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\varepsilon$
    $\mathcal{L}_{\text{noise}} \leftarrow \frac{1}{2}\|\varepsilon_\theta(x_t, t) - \varepsilon\|^2$
    $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{\text{noise}}$
  end
  return θ
end

Noise uncertainty and the conditional mean. A known training target does not mean the network can recover it exactly. The noisy point $x_t = (0, 0)$ could have come from any of the four clusters. Each possible origin requires a different noise to reach that point, and the network cannot distinguish them. To describe what remains possible after an observation, use a *conditional density*. We write it as $p(u \mid y)$ for an unknown quantity u given an observed value y. The vertical bar means "given". Here u is ε and y is x_t, at a fixed level t. The density weights the possible noise vectors according to how likely their clean origins are. A predictor must return one vector even when several are possible. Their probability-weighted average is the conditional mean, $\mathbb{E}[u \mid y]$. At the origin, symmetry gives equal weight to opposite directions, so the average noise is zero.

Squared error and the conditional mean. Let a be any prediction made from y. Its average squared error separates into two terms:

$$\mathbb{E}[\|u - a\|^2 \mid y] = \mathbb{E}[\|u - \mathbb{E}[u \mid y]\|^2 \mid y] + \|\mathbb{E}[u \mid y] - a\|^2. \tag{7}$$

The first is the uncertainty left after observing y. The second measures how far the prediction is from the conditional mean. Only the second depends on a, so choosing $a = \mathbb{E}[u \mid y]$ minimizes the error. Noise-prediction training therefore aims at $\mathbb{E}[\varepsilon \mid x_t]$ at each level. Even this optimal prediction leaves the first term of the identity, so a nonzero training loss need not signal a bad model. Nor is an average necessarily a likely individual answer: if u is equally likely to be -1 or $+1$, its mean is 0. The predictor returns an average; the sampler must still produce individual samples.

DDPM: generating samples by removing noise. The sampler shown in the middle row of Figure 2 starts from pure noise, $x_T \sim \mathcal{N}(0, I)$, and visits the levels $t = T, \ldots, 1$ with the trained network held fixed; the box below is the loop. Each step predicts the noise, then forms the mean on line 4: it subtracts the share that the last forward step added to x_t and undoes that step's shrinkage. It then draws $z_t \sim \mathcal{N}(0, I)$ and adds $\sqrt{\beta_t} z_t$, except at $t = 1$, where it returns the mean. With the exact conditional mean noise, that mean would be exact; a learned predictor approximates it. Treating the reverse distribution as Gaussian around the mean is a separate approximation, justified by small steps.

The loop over all levels is:

1: **function** DDPM-Sample(x_T, θ)
2: **for** $t = T, T-1, \ldots, 1$ **do**
3: $\varepsilon \leftarrow \varepsilon_\theta(x_t, t)$
4: $\mu \leftarrow \frac{x_t}{\sqrt{\alpha_t}} - \beta_t \frac{\varepsilon}{\sqrt{\alpha_t(1-\bar{\alpha}_t)}}$
5: **if** $t > 1$ **then**
6: $z_t \sim \mathcal{N}(0, I)$
7: $x_{t-1} \leftarrow \mu + \sqrt{\beta_t} z_t$
8: **else**
9: $x_{t-1} \leftarrow \mu$
10: **end**
11: **end**
12: **return** x_0
13: **end**

Clipping the sampling state. Replacing values below a lower bound by that bound, and values above an upper bound by that bound, is clipping. Some image samplers use it to restrict values to an interval. We omit it here: clipping to $[-1, 1]$ would remove this mixture's four centres.

KEY CONCEPTS

- **Gaussian mixture and modes:** a weighted sum of Gaussian densities; well-separated components produce distinct regions of high density.
- **Noise and diffusion level:** noise is one standard Gaussian draw per coordinate; the level counts the noising steps applied.
- **Forward diffusion and variance schedule:** each step shrinks the sample and adds noise of variance β_t, broadening and eventually merging the modes; one closed-form jump reaches any level.
- **Noise predictor and loss:** squared-error regression on the known noise learns the conditional mean noise $\mathbb{E}[\varepsilon \mid x_t]$, not the noise itself.
- **DDPM generation:** subtract one step's share of the predicted noise, then sample around that mean; keep the network fixed.
- **Mode population and per-coordinate spread:** cluster fractions and within-cluster standard deviations check different properties of the generated distribution.

3 VERIFY

(1) **Forward diffusion turns four clusters into one Gaussian.**

> Make a movie of forward diffusion showing how the modes merge. Follow the same samples as the noise level increases, with a fixed coordinate scale. Print each coordinate's mean and variance at t=1000.

The centres move toward the origin while each cloud broadens; by $t = 1000$ the four groups are indistinguishable. With at least 2000 samples, each coordinate's mean should lie within 0.1 of zero and its variance within 0.1 of one. Watch the reference forward process.

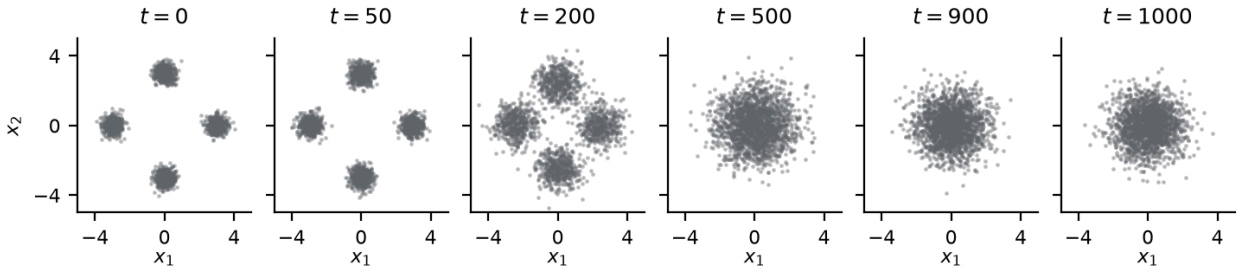


Figure 6: The forward end state is approximately standard Gaussian. The reference follows the same 2000 points through forward steps at six levels on one scale.

(2) **Training moves the generated endpoints onto the clusters.**

> Make a movie of the 10 saved training stages. At each stage, show 10 DDPM generation paths with the current predictor and the training-loss history so far. Reuse the same initial Gaussian samples and reverse-step noise across stages.

Each path runs from $t = 1000$ to 0 with fixed weights; the training stage changes the weights. Shared starts and step noise isolate the effect of learning. At initialization the paths leave the plotted square, some reaching $|x| \approx 1500$; at the final stage all ten endpoints sit on the four clusters. In the reference most of the change happens in the first few hundred of its 25000 updates, while the loss falls. The loss then levels off, near 0.25 in the reference, rather than at zero. Part 1's conditional-mean identity explains this remaining loss: the added noise is still uncertain given the noisy point. Watch the reference training process.

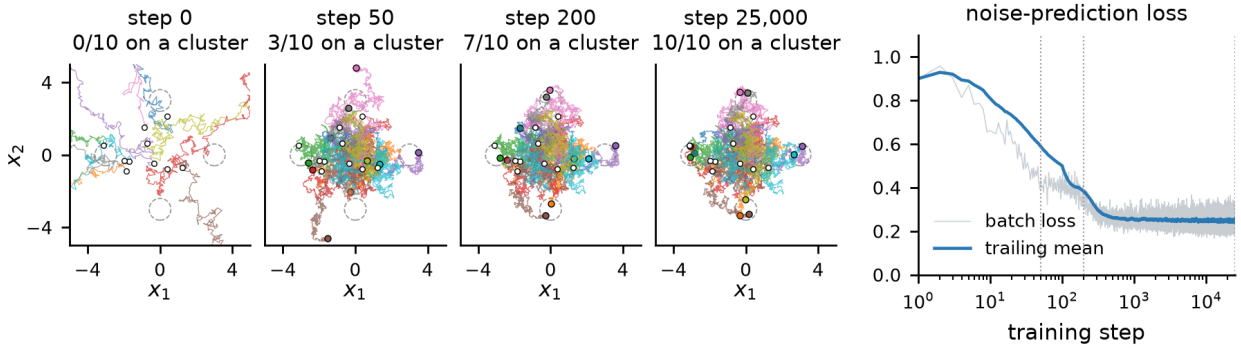


Figure 7: Endpoints move onto the four clusters as the loss falls. The same ten generation paths (white starts, coloured ends, dashed circles of radius 2σ around the centres) with the predictor saved at four of its ten training stages; dotted lines mark those stages on the loss history.

(3) **Generated samples recover the cluster populations, centres and widths.**

> Generate 2000 samples with the trained predictor and 1000 DDPM steps. Make a movie of the reverse process and compare the final distribution with the target. Plot the samples and measure each mode's centre, population and per-coordinate spread.

Assign each generated point to its nearest target centre. Your populations should lie within 0.04 of 0.25, your spreads within 0.03 of 0.30 and your centres within 0.05 of the targets. These limits allow three sampling

fluctuations of exact data, about 0.01 for both quantities at 2000 samples, plus a small training error. The reference gives populations from 0.245 to 0.262 and spreads from 0.29 to 0.30. Check all three: equal populations can hide shifted or broadened clouds, and a low loss alone does not establish the match. Watch the reference generation process.

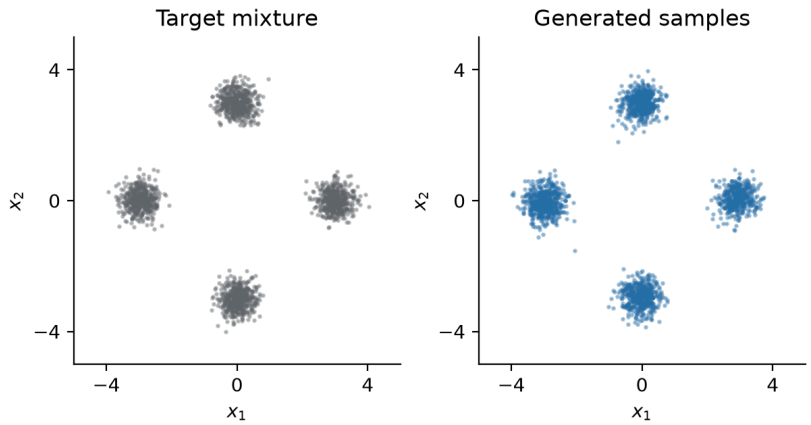


Figure 8: Generated samples recover the target's centres, equal populations and spread. The reference run's samples are plotted beside exact target samples.

Part 2: Deterministic sampling with DDIM

Goal: replace DDPM's random reverse step with a deterministic update that reuses the trained noise predictor and can jump between selected noise levels. Check that repeated runs from one starting noise agree exactly, and that the generated points still recover the four clusters.

(1) DO

Part 1's DDPM sampler evaluates the network at all 1000 levels, so each sample costs 1000 predictions. DDIM keeps the trained predictor and changes only the reverse rule. The surprising part is that after the start it makes no random draw at all and can use tens of levels instead of 1000, yet its samples still recover the four clusters. Deleting DDPM's noise term is not enough; the coefficients change too. The prompt below asks for that sampler, with the mixture's exact noise prediction for comparison.

```
Implement deterministic DDIM sampling with the trained noise predictor over a
selectable subset of noise levels. Also support the mixture's exact conditional-mean
noise prediction for comparison.
```

(2) UNDERSTAND

Part 1 generates a sample with 1000 DDPM steps, each requiring a network evaluation. Can we reduce this cost while keeping the generated distribution correct? DDIM changes the sampling rule while keeping the trained predictor. We will construct its update from Part 1's noising formula, then measure the accuracy lost when we skip levels.

Estimating the original point with a denoiser. Part 1's DDPM update estimates the next state at level $t-1$ and samples around it. To make a larger jump, DDIM first estimates x_0, the original point before diffusion noise was added. A function that estimates this clean point from x_t is a *denoiser*. Several clean points could have produced the same noisy point. Part 1's squared-error identity says their conditional mean $\mathbb{E}[x_0 \mid x_t]$ gives the smallest average squared error. This mean need not match the original point in any particular training example. To estimate it, solve Equation 5 for x_0 and substitute the predicted noise for the unknown noise. This use of *Tweedie*'s formula subtracts the estimated noise and undoes the forward shrinkage (line 4 of the box). With the exact conditional-mean noise prediction, it returns the exact conditional mean clean point.

The deterministic DDIM update. Apart from the levels it visits, the only difference from Part 1's sampler is the reverse update: the loop and the noise estimate are the same. Let $t' < t$ be the next noise level visited. DDIM combines the clean estimate with the predicted noise in the proportions for level t' (line 5 of the box). The coefficients use the destination level, which need not be $t-1$; choosing fewer destination levels means fewer network evaluations.

The DDIM sampler, with the differences from Part 1 in red:

function DDIM-SAMPLE(x_T, θ, levels)
  for each jump $t \to t'$ in levels **do**
    $\varepsilon \leftarrow \varepsilon_\theta(x_t, t)$
    $\hat{x}_0 \leftarrow \frac{x_t}{\sqrt{\overline{\alpha}_t}} - \varepsilon\sqrt{\frac{1}{\overline{\alpha}_t} - 1}$
    $x_{t'} \leftarrow \sqrt{\overline{\alpha}_{t'}}\hat{x}_0 + \sqrt{1 - \overline{\alpha}_{t'}}\varepsilon$
  end
  return x_0
end

Red marks the lines that differ from Part 1's DDPM sampler: the levels visited, the estimate and the update. Part 1's update forms the posterior mean and adds fresh Gaussian noise around it; this one reuses the predicted noise without a draw. Deleting DDPM's random term alone is not enough: the coefficients change too. Randomness enters only in the starting draw.

Errors from large steps and learned predictions. A prediction applies at the current point. Over a large jump, it may no longer describe the whole path accurately. DDIM therefore does not guarantee correct samples at every step count. DDPM can also skip levels, so we compare both on the same *grid*, the list of levels visited. A different grid gives a different trajectory from the same starting point. Their generated distributions determine which needs fewer evaluations.

To isolate errors in the learned prediction, repeat the same DDIM update using the *exact* conditional-mean noise, calculated from the known mixture rather than approximated by the network: $\varepsilon_{\text{exact}}(x_t, t) = \mathbb{E}[\varepsilon \mid x_t]$. Compute the conditional mean noise within each component, then average these means using the probabilities of the components given x_t. This removes predictor error while retaining the chosen steps.

DDPM with skipped levels. The step-count comparison also needs a random sampler on the coarser grid. For a jump from t to t', set $a = \overline{\alpha}_t / \overline{\alpha}_{t'}$. The forward jump shrinks by $\sqrt{a}$ and adds noise of variance $1 - a$. Use Part 1's reverse rule with these effective coefficients:

$$\mu_{t \to t'} = \frac{1}{\sqrt{a}}\left(x_t - \frac{1 - a}{\sqrt{1 - \overline{\alpha}_t}}\varepsilon_\theta(x_t, t)\right). \tag{8}$$

Add Gaussian noise of variance $1 - a$ when $t' > 0$, and none when $t' = 0$. For adjacent levels this is exactly Part 1's choice of variance β_t. The K-step comparison visits K evenly spaced levels before reaching zero.

KEY CONCEPTS

- **Tweedie's formula and DDIM:** estimate the clean point from the predicted noise, then move to a selected lower level with no fresh draw.
- **Denoiser and conditional mean:** the estimate targets the average clean point over the possible originals, not the particular training sample.
- **Skipped levels and the grid:** the destination level need not be $t - 1$; a coarser grid costs fewer evaluations but makes larger steps.
- **Deterministic step:** the update fixes the next point; randomness enters only in the starting draw.

3 VERIFY

(1) **DDIM's reverse step is deterministic; DDPM's is not.**

> Run DDIM over 1000 levels twice from the same noise, and DDPM from the same starting noise with two noise draws. Plot their trajectories and 2000 samples of each beside exact samples. Print the largest difference between the two DDIM runs.

The two DDIM runs agree exactly; the reference prints 0. Across the 2000 shared starts, the two DDPM runs end in the same cluster only about a quarter of the time, as chance predicts for four clusters. Both samplers still reproduce the target: DDIM cluster fractions lie within 0.015 of 0.25, and both samplers' widths lie between 0.29 and 0.32.

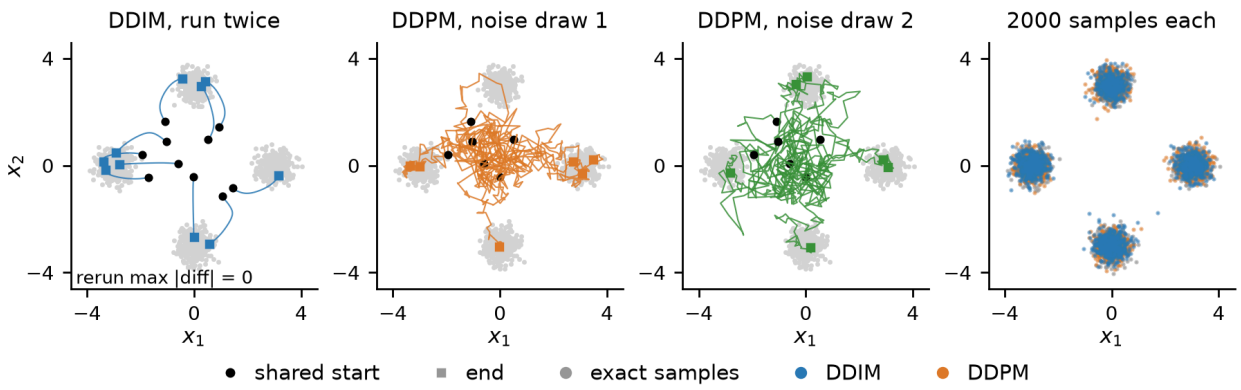


Figure 9: One starting noise gives one deterministic sample; the stochastic sampler does not, yet both reproduce the target distribution.

Part 3: Flow matching and the cost of fewer steps

Goal: train a velocity network on straight paths by flow matching, check it against the mixture's exact marginal velocity, and compare DDPM, DDIM and flow matching on one step grid. Measure how many steps each sampler needs to reach the target.

(1) DO

Part 2's DDIM sampler reuses the trained noise predictor; this Part trains a second network to predict the motion itself. It learns from straight lines between a data point and a random point, lines that no real sample follows. The surprising part is that following its predictions for about 20 steps still produces samples from the four clusters, less than half the steps DDPM needs, even though the network never saw a real generation path. The prompts below ask for the velocity network and for that comparison.

```
Train a velocity network on the same mixture by flow matching, with the path x_s=(1-
s)x_0+s z, z standard Gaussian, s uniform in [0,1], and target velocity z-x_0. Use the
noise predictor's architecture, trained from scratch, and a learning rate that decays
to zero. Sample by Euler steps from s=1 to s=0. Compute the exact marginal velocity of
the mixture for comparison.
```

```
Compare DDPM, DDIM and flow-matching samples with 1,2,5,10,20,50,100 steps, and 1000
levels for DDPM and DDIM. Fewer steps use evenly spaced levels. Use 4000 samples per
run. Record cluster fractions, centres and widths, and the energy distance to 4000
exact samples. Estimate its null scale from 20 pairs of independent exact sample sets.
```

(2) UNDERSTAND

Flow matching on straight paths. DDIM reuses a network trained to predict noise. Could we instead train a network to predict motion directly? We need examples with known velocities. Join a data point x_0 and an independent Gaussian point z by a straight path. These are the path's two endpoints. Time s runs continuously from 0 to 1:

$$x_s = (1-s)x_0 + sz, \quad \frac{dx_s}{ds} = z - x_0. \tag{9}$$

Each pair supplies a position and its known velocity at every s. Flow matching trains a network on these constructed examples, just as Part 1 trained on examples with known noise.

The velocity network model. The network receives x_s and s and returns a velocity estimate $v_\theta(x_s, s)$. It reuses the noise predictor's architecture and output shape, but starts with newly initialized weights rather than the noise predictor's trained weights.

The flow-matching loss. The target is the known path velocity $z - x_0$. The box's loss line is the mean squared error over independent data, Gaussian draws and s. Drawing s uniformly in $[0, 1]$ gives equal-length time intervals equal probability.

Training the velocity network. The loop below draws a batch, evaluates the loss and updates the weights, as in Part 1.

function TRAIN-VELOCITY(θ)
  for each update **do**
    $x_0 \sim p_{\text{data}}$
    $z \sim \mathcal{N}(0, I)$
    $s \sim \mathcal{U}[0, 1]$
    $x_s \leftarrow (1 - s)x_0 + sz$
    $\mathcal{L}_{\text{flow}} \leftarrow \|v_\theta(x_s, s) - (z - x_0)\|^2$
    $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{\text{flow}}$
  end
  return θ
end

The marginal velocity. Different pairs can pass through the same position at the same time with different velocities. The network cannot identify the pair from (x_s, s) alone. By Part 1's squared-error identity, it learns their conditional average, the marginal velocity:

$$v(x, s) = \mathbb{E}[z - x_0 \mid x_s = x]. \tag{10}$$

Here "marginal" means that we have averaged over the unobserved endpoints x_0 and z, which the network does not receive.

Following a velocity field. A rule assigning a velocity $v(x, s)$ to every position x and time s is a *velocity field*. A moving point follows $\frac{dx_s}{ds} = v(x_s, s)$. This is an *ordinary differential equation* (ODE): it specifies the rate of change of a point along its path. Following the field moves the whole cloud of points, so we must check how its density changes.

Probability flux and conservation. Let $\rho_s(x)$ be the moving cloud's density. The product $\rho_s v$ is the *probability flux*, the amount of probability crossing a boundary per unit area per unit time. A region gains probability when more enters than leaves. For any vector field w, its *divergence* $\nabla_x \cdot w = \sum_i \partial w_i / \partial x_i$ adds the coordinate rates of change. For a flux it measures the net amount leaving a small region per unit volume. Here $\partial w_i / \partial x_i$ is a derivative with respect to one coordinate, holding the others fixed. Positive divergence of the probability flux lowers the density. This gives the *continuity equation*, which states conservation of probability:

$$\partial_s \rho_s + \nabla_x \cdot (\rho_s v) = 0. \tag{11}$$

Here ∂_s is the derivative with respect to time. This is Week 4's mass conservation with probability in place of fluid.

Transport by the average velocity. Why does this averaging preserve the wanted distribution? Each constructed path carries probability with its own velocity. Adding their probability fluxes gives the density at (x, s) times the conditional average velocity there. Thus this velocity satisfies Equation 11 for the distribution along the constructed paths. Following it transports that distribution even though an individual trajectory need not follow one original straight segment.

The exact mixture velocity and its error. To check the learned velocity, calculate its conditional average separately within each Gaussian component. Both endpoints are Gaussian, so each component's average velocity is a constant multiple of x plus a constant vector. These constants depend on s. At (x, s), weight each component's velocity by its probability of producing that position. These probabilities are proportional to the component densities there; divide by their sum to normalize them. Averaging the component velocities gives the mixture's exact marginal velocity. To compare vector predictions, square the length of each error, average over points, and take the square root. This is the *root-mean-square* (*RMS*) error. Divide it by the RMS length of the exact velocity to obtain the relative error. Squaring before averaging prevents opposite errors from cancelling.

Euler sampling with the learned velocity. Generation starts from Gaussian noise at $s = 1$ and integrates backward with Euler steps, holding the velocity constant over one step of size $\Delta s > 0$:

$$x_{s-\Delta s} = x_s - \Delta s \, v_\theta(x_s, s). \tag{12}$$

The training segments are straight, but trajectories of their averaged field can curve, so accuracy at few steps is a measured result, not a consequence of the construction.

Comparing sample distributions with energy distance. The cluster measurements check one property at a time. The *energy distance* compares whole distributions using distances between pairs of points. No physical energy enters this statistical measure. Compare generated samples with the exact draws from the target introduced in Part 1. Let X and Y be independent draws from the two distributions. Primes denote further independent draws, X' and Y'. The energy distance is

$$D = 2\mathbb{E}\|X - Y\| - \mathbb{E}\|X - X'\| - \mathbb{E}\|Y - Y'\|. \tag{13}$$

The first term measures separation between the distributions. The other two subtract the spread already present within each distribution. The exact value is nonnegative and is zero exactly when the distributions agree.

The null scale. Finite samples fluctuate even when both sets come from the target. Replacing the expected distances with averages over distinct sample pairs can give a slightly positive or negative estimate of D. To measure this sampling variation, compare independent pairs of exact sample sets. Their distances define the *null scale*. We summarize it by the 95% *quantile*, the value below which 95% of these distances fall. A generated sample set within this scale shows no difference large enough for this measure to distinguish at the chosen sample size.

Sampling variation of mode fractions. Even exact draws do not put exactly a quarter of a finite sample in each cluster. Repeated sets of N independent points give slightly different fractions. The standard deviation of such an estimated quantity is its *standard error*. For a cluster with probability p, the fraction's standard error is $\sqrt{p(1-p)/N}$. The reference plot shades two standard errors around $p = 0.25$. This is the *binomial* sampling variation, the variation in how many of N independent draws enter one chosen cluster.

Further reading: the Flow Matching Guide, Sections 4.2–4.5, for conditional paths and their averaged velocity, and Lipman et al. for flow matching itself; Stable Diffusion 3 trains its image generator this way. The guide runs time from noise to data; we keep data at $s = 0$.

KEY CONCEPTS

- **Flow matching and marginal velocity:** regression on straight-line velocities learns their conditional mean, which Euler steps follow from noise to data.
- **Probability conservation:** the averaged path velocities carry the same probability flux as the constructed training paths.
- **Energy distance and null scale:** one number for two whole sample sets, judged against its spread between exact sets.

3 VERIFY

(1) **The learned velocity matches the exact marginal velocity.**

> Plot the learned and exact velocity fields at s=0.1,0.5,0.9 on a grid over the samples, and report their relative RMS difference at each s, the RMS of the difference divided by the RMS of the exact velocity.

The arrows overlap; yours should differ by less than 0.1 at every s, and the reference gives 0.011, 0.017 and 0.058. The arrows show dx_s/ds for increasing s; generation runs against them. Near $s = 1$ the field approaches $v(x, 1) = x$, because the mixture's mean is zero.

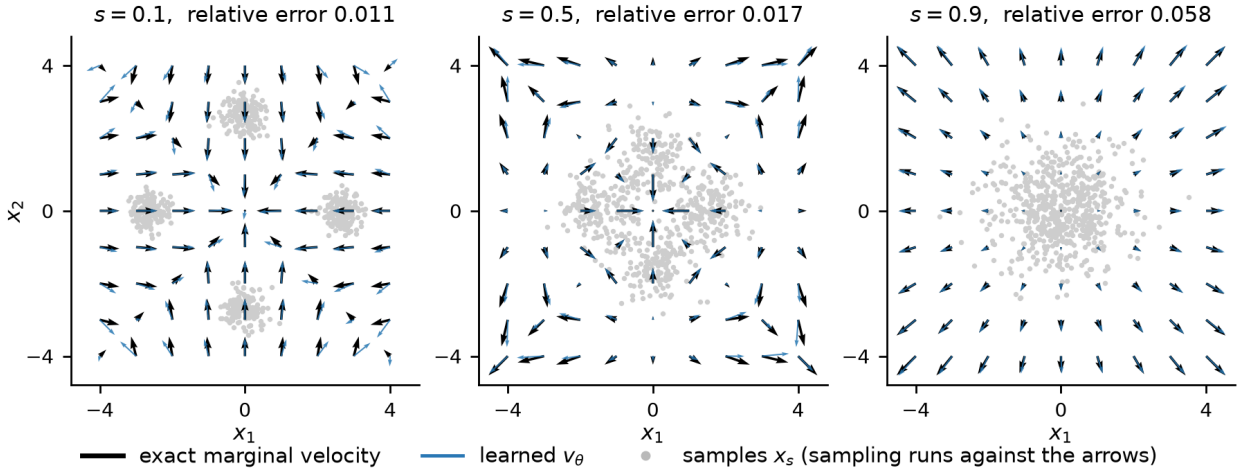


Figure 10: The learned field reproduces the mixture's marginal velocity, which is why Euler steps can follow it. Arrow lengths are clipped in the far field.

(2) **Flow matching and DDIM need tens of steps; DDPM needs more.**

Plot the energy distance, the largest cluster-fraction error and the largest width error against the number of steps for the three samplers, with the null scale shaded.

By 20 steps your flow sampler should reach an energy distance below 0.005 and widths within 0.03 of 0.30. The reference flow sampler reaches 0.0015 at 20 steps, at the edge of the null 95% quantile 0.0013; DDIM reaches 0.0038, and DDPM needs about 50 levels. One or two steps fail for every sampler, even with the exact noise prediction or velocity: the samples collapse toward the mean. On this mixture the gain over DDIM is modest.

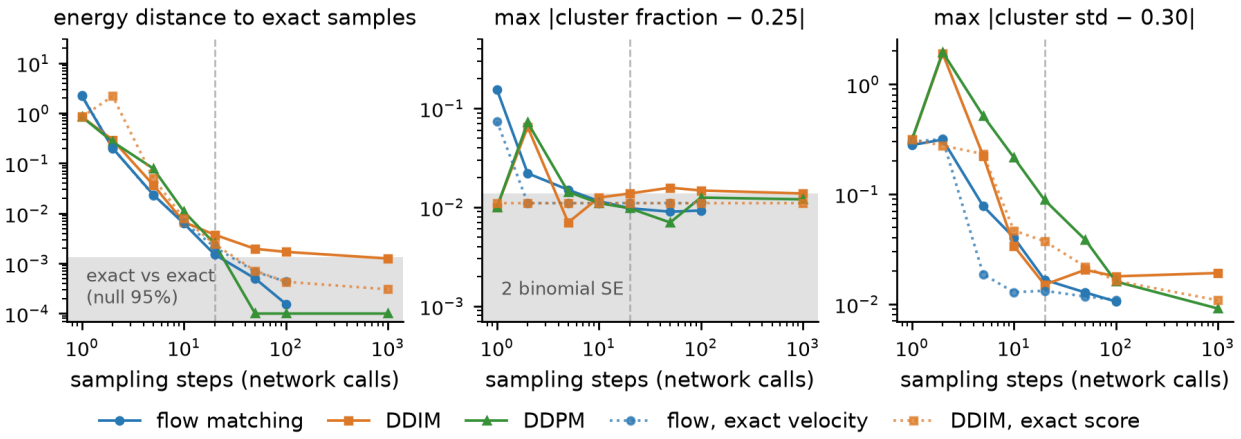


Figure 11: Flow matching and DDIM reach the target in tens of steps while DDPM needs about fifty. Shaded bands are the exact-versus-exact null scale and two binomial standard errors of a cluster fraction; energy distances below 10^{-4} are drawn at 10^{-4}. The dashed line marks 20 steps.

Part 4: The same predictor on photographs

Goal: load a predictor already trained on CIFAR-10 photographs and generate recognizable images with Part 1's DDPM update, without further training. Part 5 uses the same model to fill in missing pixels.

1 DO

The weights are someone else's; the sampler is yours. On a laptop CPU, Part 1's update turns noise into recognizable photographs. Download the CIFAR-10 noise predictor, already trained, from Hugging Face. Keep these files for Part 5.

```
uv run --with huggingface-hub==1.33.0 hf download google/ddpm-cifar10-32 \
    config.json diffusion_pytorch_model.safetensors \
    --revision 267b167dc01f0e4e61923ea244e8b988f84deb80 \
    --local-dir inputs/cifar-model
```

Sampling, not loading, is the cost: the reference spends about 0.05 seconds per image per network evaluation on its CPU, so 1000 updates on four images take roughly three minutes.

Load the downloaded pretrained weights and configuration as a fixed noise predictor in JAX. Match the network in this reference implementation: https://github.com/huggingface/diffusers/tree/v0.40.0

Use the pretrained predictor with the same 1000-step DDPM sampler as on the mixture, without clipping during sampling. Use float32 arrays and pass noise level t as model index t-1. Keep the downloaded weights fixed.

2 UNDERSTAND

The pretrained predictor and its files. We use `google/ddpm-cifar10-32`, a network trained to predict noise added to CIFAR-10 photographs. A pretrained network has already learned its weights, so we can use it without repeating the training. It is *unconditional*: it receives no requested object category or text description. The weights file supplies the learned values θ from the image model's training. They work only with that model's architecture. The *configuration*

file describes that architecture so we can reconstruct it. The *revision* in the download command selects a fixed version of both files. We download the predictor; our sampler still computes the reverse updates and keeps the downloaded weights fixed.

The image predictor's inputs and output. As in Part 1, the network receives a noisy sample x_t and its level t. Here the sample is a 32×32 image. Each pixel has a red, green and blue value; the three grids of values are its *colour channels*. The image is stored as an *array*, an indexed collection of numbers. Its *shape* lists the number of entries along each direction, here $32 \times 32 \times 3$. The output $\varepsilon_\theta(x_t, t)$ has the same shape, with one estimated noise value per pixel and colour channel.

Convolutions and feature channels. To estimate noise, the network first extracts local patterns from the image. A *filter* is a small grid of learned weights. Multiply these weights by a nearby group of image values and add the results to get one response. Sliding the filter over the image repeats this calculation at each position; this operation is a *convolution*. The resulting grid of responses is a *feature channel*. The input has three colour channels; the first convolution produces 128 feature channels. These internal channels are learned measurements, not colours.

The U-Net's down path, up path and skip connections. A small local pattern may be ambiguous without its surroundings. The network gathers wider context by processing successively smaller grids, with fewer positions along each side. This grid size is its spatial *resolution*. Our model uses 32×32, 16×16, 8×8 and 4×4 grids. Each position on a smaller grid can combine information from a larger region of the original image. The output still needs one noise estimate at every original pixel, so the network enlarges the grid back to 32×32. The shrinking and enlarging stages form the down and up paths of a *U-Net*, named for the U-shaped drawing in Figure 12. Shrinking can lose fine spatial information. *Skip connections* carry saved feature values directly from the down path to the up path at matching resolutions. The up path combines this detail with the information arriving from smaller grids.

The operations in the original U-Net figure. In Figure 12, "conv" means convolution. Blue arrows use 3×3 filters, then set negative values to zero with the *rectified linear unit* (ReLU). Its slope changes at zero, so no single straight line describes this *nonlinear* operation. Red arrows keep the maximum in each 2×2 group, shrinking the grid; this is *max pooling*. Green arrows enlarge the grid with a learned filter, the *up-convolution*. Grey arrows copy saved features and *crop* them, trimming the edges so the grids fit together. The final 1×1 convolution combines feature channels at each position to predict a category for each pixel. Assigning pixel categories is *segmentation*, the original network's task. The input "tile" is a rectangular piece of an image. Our predictor instead estimates noise in each colour channel.

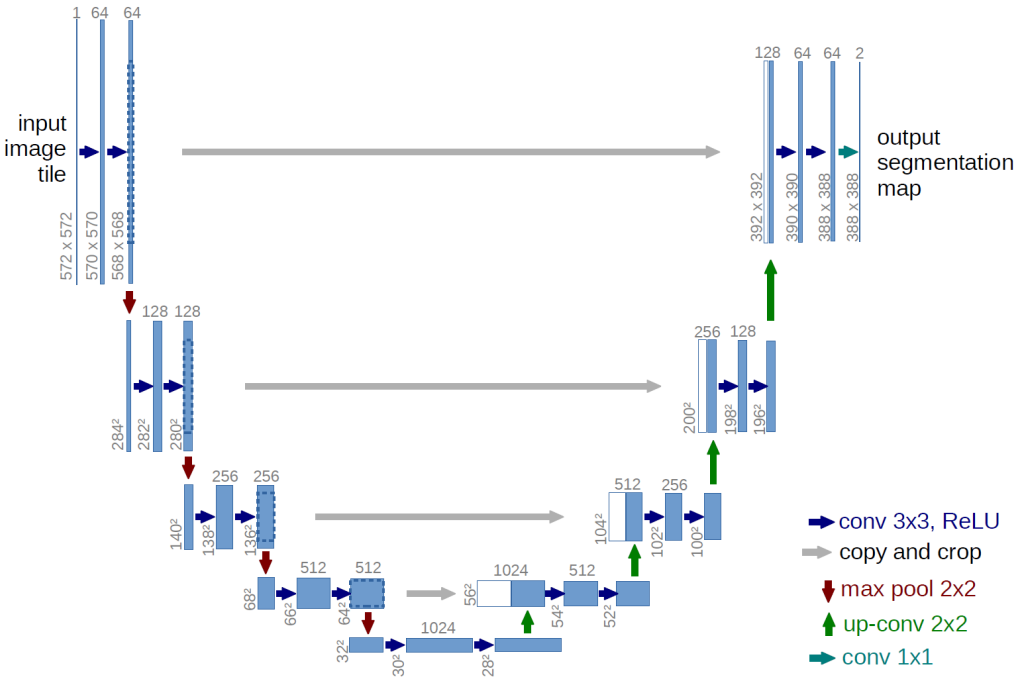


Figure 12: The original U-Net architecture, Figure 1 of Ronneberger, Fischer and Brox [4], reproduced from the authors' page. Follow the down path, also called the contracting path, on the left and the up or expanding path on the right. Numbers above the blue boxes count feature channels; numbers beside them give grid sizes. This is the original segmentation network, not the downloaded noise predictor. Its dimensions and output differ.

The downloaded model's grids and time input. The literature figure explains the U-shape, not our model's exact settings. Our downloaded model uses side lengths $32 \to 16 \to 8 \to 4 \to 8 \to 16 \to 32$. Its down-path channel counts are $128, 256, 256, 256$. It uses learned convolutions to shrink the grid, rather than the original figure's maximum operation. The input and output both have three colour channels. The full network has about 35.7 million parameters.

The predictor also receives t. It converts the level into a list of numbers, the *time embedding*, and supplies it throughout the network. This serves the same purpose as Part 1′s time features: predictions depend on the noise level. One evaluation produces one noise estimate for the sampler's next reverse step.

Image sampling and display conventions. Replace Part 1′s predictor with the downloaded one and apply the same reverse update through all 1000 levels. The sampler now updates an image-shaped array. The model uses *float32*, a format storing each number approximately with 32 binary digits. The model selects its noise level by an *index* starting at zero. Pass our level t as model index $t - 1$ to align the prediction with the sampler's coefficients. Clean colour values use the range $[-1, 1]$. For display, map a value x to $\frac{x+1}{2}$ and clip it to $[0, 1]$. This changes only the displayed image; keep the sampling states unclipped, as in Part 1.

KEY CONCEPTS

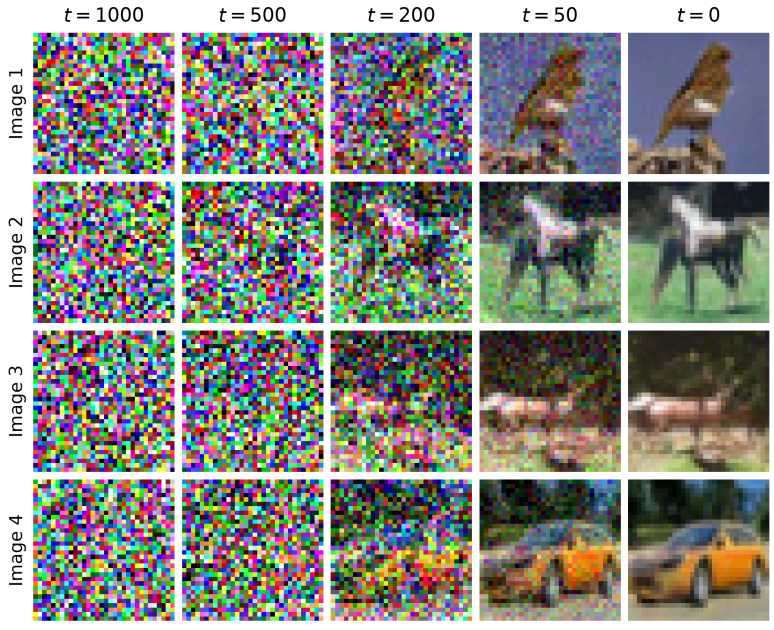
- **Weights and configuration:** learned parameter values and the architecture needed to use them.
- **Noise prediction:** noisy image and noise level in, an image-shaped noise estimate out.
- **Generation:** Part 1′s DDPM update, unchanged, turns that estimate into an image.

③ VERIFY

(1) **The downloaded predictor generates recognizable photographs.**

```
Generate four images from Gaussian noise using the downloaded predictor and the
1000-step DDPM sampler. Visualize stages of the reverse process and the final
images. Compare them with real CIFAR-10 images using the Week 6 image-gallery
scripts.
```

Follow each noise image through the reverse process and compare the emerging colours, shapes and objects with the CIFAR-10 examples of Week 6′s Part 1. Structure appears late: at $t = 200$ the images are still mostly noise, and objects become recognizable only between $t = 200$ and $t = 50$. Explain which part of this process was learned before the download and which part your sampler computes now.



Pretrained noise predictor; the same DDPM update, all 1000 steps.
Recorded states; display clips to [−1, 1]. Native 32 × 32 pixels.

Figure 13: The downloaded weights and Part 1′s update produce recognizable objects without further training. Each row follows one sample; states are clipped only for display.

Part 5: Filling in missing pixels with the downloaded model

Goal: use the downloaded predictor to fill in a hidden square of a photograph, and check that every filled image keeps the visible pixels exactly. Repeating the process with different random choices should fill the hole with different, plausible content.

1 DO

The sampler never receives the hidden pixels, yet it fills the hole; the visible pixels stay exact because every merge overwrites them. Fetch the prepared CIFAR-10 images, stored in $[-1, 1]$, into your `week7/` directory. The file holds four test images, which Part 5 uses.

```
mkdir -p inputs
curl -fL https://giggleliu.github.io/AMAT5315-2026Fall/downloads/week7-images.npz \
  -o inputs/images.npz
```

```
With the pretrained CIFAR-10 predictor, inpaint images from observed pixels and a
mask, given the observations and the mask only. Implement RePaint with 50 reverse
jumps over levels 1000,980,...,20,0, two attempts per jump except the last, forward
noise between attempts, and known pixels replaced by forward-noised observations.
```

```
Use the four test images in inputs/images.npz. Hide rows and columns 6:26, all three
channels, and inpaint each image with RePaint using seeds 7201,7202,7203,7204. Record
each completion and its hidden-pixel mean squared error (MSE).
```

The study makes 16 completions of 99 network evaluations each, a few minutes on a laptop CPU.

2 UNDERSTAND

Inpainting and the observation mask. We now want to fill the missing part of an image, a task called inpainting. The filled image is a completion. It must preserve the visible pixels and fill the hole with content that fits them. The visible values are the *observations*; the sampler does not receive the hidden truth. A mask M with the image's shape specifies which values may change. It contains 1 at observed entries and 0 at hidden ones. Our mask hides a central 20×20 square in all three colour channels. Write Y for the observed image values where $M = 1$.

Observed pixels at the sampler's noise level. Keeping chosen values fixed is *clamping*, as in Week 6's restricted Boltzmann machine (RBM). Here the sampler's intermediate image X_t is noisy. Pasting unnoised observed pixels into it would create an image clean in one region and noisy in the rest, which the network never saw in training. First add noise to the observed values at the same level, using Part 1's direct noising formula:

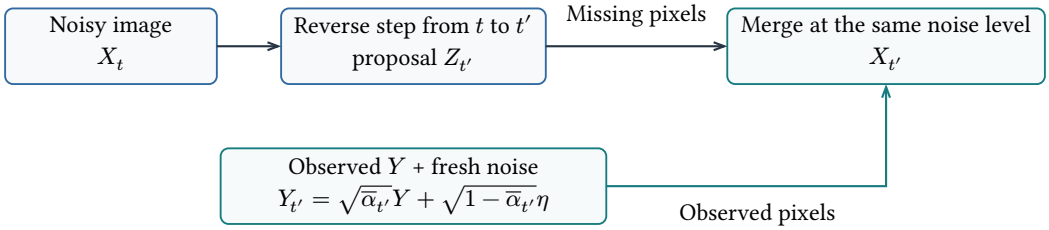
$$Y_{t'} = \sqrt{\overline{\alpha}_{t'}}\,Y + \sqrt{1 - \overline{\alpha}_{t'}}\,\eta, \quad \eta \sim \mathcal{N}(0, I). \tag{14}$$

These are the forward-noised observations.

The RePaint merge. RePaint is an inpainting sampler that replaces the observed pixels after every reverse step and can retry steps to improve the fit across the hole's boundary. Call the image suggested by a reverse step its *proposal*, $Z_{t'}$. Keep its hidden pixels, but replace its observed pixels with $Y_{t'}$. This combination is the *merge*, written using the mask as

$$X_{t'} = M \odot Y_{t'} + (1 - M) \odot Z_{t'}, \tag{15}$$

where $\odot$ multiplies matching entries. At observed entries the formula selects $Y_{t'}$; at hidden entries it selects $Z_{t'}$. At $t' = 0$ the forward-noised observation is the clean observation Y. The last merge therefore reproduces every observed value exactly. It leaves the proposed hidden region unchanged.



$$X_{t'} = M \odot Y_{t'} + (1 - M) \odot Z_{t'}$$

Figure 14: Observed pixels are reproduced exactly. The diagram traces one RePaint step: merge the forward-noised observation into the generated state at the same level.

RePaint's reverse proposal distribution. To reduce the number of network evaluations, RePaint visits $1000, 980, \ldots, 20, 0$. Each proposal jumps from t to $t' < t$, skipping levels as in Part 2. We use Part 2's Tweedie estimate $\hat{X}_0$, clipped to the clean image range $[-1, 1]$. Only this estimate is clipped, not the noisy state. Set $a = \frac{\overline{\alpha}_t}{\overline{\alpha}_{t'}}$ as in Part 2. If we treat the clean estimate as the original image, the Gaussian forward rule gives a conditional distribution for the lower-level state. Its mean is

$$\mu_{t\to t'} = \frac{\sqrt{\overline{\alpha}_{t'}}(1-a)}{1-\overline{\alpha}_t}\hat{X}_0 + \frac{\sqrt{a}(1-\overline{\alpha}_{t'})}{1-\overline{\alpha}_t}X_t. \tag{16}$$

It combines the clean estimate with the information still in the current noisy state. RePaint uses the variance of that same Gaussian conditional distribution:

$$v_{t\to t'} = \frac{(1-\overline{\alpha}_{t'})(1-a)}{1-\overline{\alpha}_t}. \tag{17}$$

This differs from Part 2's choice $1 - a$ for the DDPM comparison. Draw the proposal $Z_{t'}$ around $\mu_{t\to t'}$ with this variance per coordinate, then apply the RePaint merge. At $t' = 0$ the variance is zero.

Repeating a jump after the merge. The merge replaces observed pixels after the network has proposed the hidden region. A second attempt lets the network respond to the newly merged values: the box's forward-noise line returns to t, then repeats the proposal and merge.

Use two attempts for each jump except the final jump to zero, which is taken once. The cost is $49 \times 2 + 1 = 99$ network evaluations. See Lugmayr et al., *RePaint*, Section 4.1.

function RePaint-Inpaint(X_T, Y, M)
  for $t = 1000, 980, \ldots, 40$ **do**
    $t' \leftarrow t - 20$
    for $r = 1, 2$ **do**
      if $r = 2$ **then**
        $f \sim \mathcal{N}(0, I)$
        $X_t \leftarrow \sqrt{a}X_{t'} + \sqrt{1-a}f$
      end
      $Z_{t'} \sim \mathcal{N}(\mu_{t\to t'}, v_{t\to t'})$
      $X_{t'} \leftarrow M \odot Y_{t'} + (1 - M) \odot Z_{t'}$
    end
  end
  $Z_0 \sim \mathcal{N}(\mu_{20\to 0}, 0)$
  $X_0 \leftarrow M \odot Y + (1 - M) \odot Z_0$
  return X_0
end

Hidden-pixel error and variation between completions. We know the truth inside the hole even though the sampler does not. The *mean squared error* (MSE) averages squared differences from that truth. Our *hidden-pixel MSE* uses only the hidden values, 400 pixels with three channels each, giving 1200 values. It measures recovery of the particular image that was hidden. Many photographs can agree with the same visible border, so a plausible completion can differ from that truth. A *random seed* selects the sequence of random numbers used by a run; changing it gives another possible completion. At each pixel, compute the variance across completions to show where they disagree. This *pixelwise variance* measures the sampler's variation at each location, not its error against the withheld image.

KEY CONCEPTS

- **Mask and inpainting:** the mask marks observed pixels; a completion keeps them and fills the rest.
- **Forward-noised observations:** the observed pixels, noised to the sampler's current level with Part 1's jump formula.
- **RePaint:** a coarse reverse sampler with the observed pixels overwritten after every jump; retry each jump after forward noising, except the final jump to zero.
- **Hidden-pixel MSE and the spread of completions:** error against the withheld truth; different seeds give different plausible completions where the observation says nothing.

③ VERIFY

(1) **Completions keep the observed pixels and disagree inside the hole.**

```
For each test image, show the truth, the observation, the four completions and the
pixelwise variance across them. Print each completion's hidden-pixel MSE and the
largest change of an observed pixel.
```

Every observed pixel should change by less than 10^{-5}: RePaint overwrites them at the last level. Outside the hole the four completions are identical; inside it they differ. In the reference the hidden-pixel MSE averages 0.24 over images and seeds; on the ship below it ranges from 0.13 to 0.38 across seeds. Locate the disagreements on the variance map, and explain why a completion can look plausible and still be wrong.

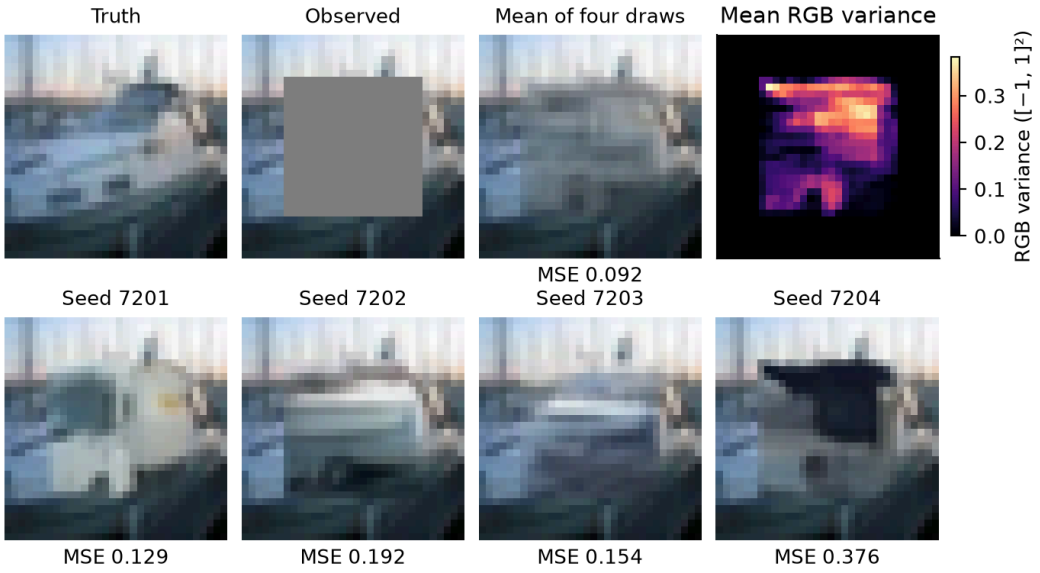


Figure 15: The observation leaves the hidden square undetermined, so each seed completes it differently. The reference ship image: truth, observation, the mean and pixelwise variance of four RePaint completions, and the four completions with their hidden-pixel MSE.

Save the results and prepare to show your work

```
Update README.md with each evidence figure beside its producing command. Explain the
transport comparison, the generated photographs and the inpainting result. Record
training settings, environment setup and the pinned model download. Keep downloaded
weights out of git. Push the source, environment and evidence.
```

Show the completed evidence at the start of Week 8. Nothing is submitted. The final project is defended in Week 13.

[1] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli, "Deep Unsupervised Learning using Nonequilibrium Thermodynamics," in *International Conference on Machine Learning*, 2015. [Online]. Available: https://arxiv.org/abs/1503.03585

[2] Y. Song and S. Ermon, "Generative Modeling by Estimating Gradients of the Data Distribution," in *Advances in Neural Information Processing Systems*, 2019. [Online]. Available: https://arxiv.org/abs/1907.05600

[3] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *Advances in Neural Information Processing Systems*, 2020. [Online]. Available: https://arxiv.org/abs/2006.11239

[4] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Medical Image Computing and Computer-Assisted Intervention*, 2015, pp. 234–241. doi: 10.1007/978-3-319-24574-4_28.